

```
2 inputs, 2 outputs, 3 product terms.
s !a b
s a !b
c a b
.p 3
01 1 0
10 1 0
11 0 1
eqntott 9,0
```

The PLA connectivity summary can be displayed using '-S' key code option. When used with 'and.eqn' example as:

```
$ eqntott -S and.eqn
```

The output will be:

```
PLA Connectivity Summary
#pterm input
1 b
1 a
#pterm output
1 c
#out #in product term expression
1 2 a & b
```

'-s' option allows you to use an output variable in another expression. Consider, for example:

```
NAME = s;
INORDER = a b;
OUTORDER = d;
c = a | b;
d = !c;
```

When you run the above example with '-s' option as:

```
$ eqntott -l -s s.eqn
```

The output will be:

```
.i 3
.o 2
.na s
.ilb a b c
.ob d c
.p 3
00-1 0
-1- 0 1
1-- 0 1
.e
```

When both the inputs 'a' and 'b' are zero, the output 'c' is zero and 'd' is '1'. If either 'a' or 'b' is '1', 'c' is '1' and 'd' is '0'.

'-f' option allows an input to also be present in the output, and used as though the value was observed at different times. Consider, for example:

```
NAME = f;
INORDER = a;
```

```
OUTORDER = a;
a = !a;
```

When you execute the above example as:

```
$ eqntott -l -f f.eqn
```

The output will be:

```
.i 1
.o 1
.na f
.ilb a
.ob a
.p 1
.f 1 1
0 1
.e
```

The truth table output is '1' only when the input is '0'. The '-s' and '-f' options are mutually exclusive.

eqntott can attempt to minimise the logic terms in equations. '-r' option will try to reduce the minterms in the output. Consider the example:

```
NAME = reduce;
INORDER = x y;
OUTORDER = z;
z = x & y | x;
```

When you use the above example with '-l' option as:

```
$ eqntott -l reduce.eqn
```

The output will be:

```
.i 2
.o 1
.na reduce
.ilb x y
.ob z
.p 2
1- 1
11 1
.e
```

Using '-r' option, eqntott will try to minimise the truth table output:

```
$ eqntott -l -r reduce.eqn
```

The output will be:

```
.i 2
.o 1
.na reduce
.ilb x y
.ob z
.p 1
1- 1
.e
```

The output 'z' is '1' whenever 'x' is '1'.

You can define expressions using 'define' keyword. The XOR logic takes

two binary inputs, and returns '1' only when either input is '1' but not both. It can be expressed as:

```
#define xor(a,b) (a&!b | !a&b)
NAME = xor;
INORDER = a b;
OUTORDER = c;
c = xor (a,b);
```

On running the example as:

```
$ eqntott -l xor.eqn
```

The output will be:

```
.i 2
.o 1
.na xor
.ilb a b
.ob c
.p 2
01 1
10 1
.e
```

As a final example, consider generating an odd parity bit for three inputs. The parity bit is set to '1' when the number of '1's in the input is even, so as to make the total number of '1's odd. We can define the logic using:

```
#define xor(a,b) (a&!b | !a&b)
NAME = parity;
INORDER = x y z;
OUTORDER = p;
p = !( xor (xor(x,y), z) );
```

When you run the above example with '-l' and '-s' options as:

```
$ eqntott -l -s parity.eqn
```

The output will be:

```
.i 3
.o 1
.na parity
.ilb x y z
.ob p
.p 4
000 1
011 1
101 1
110 1
.e
```

You can refer the manual page of eqntott for more documentation. Sources are available from the eqntott webpage. ●

The author is a free software developer at the Fedora project, and also a blogger. He also co-maintains the Fedora Electronic Lab project